

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO LAB 2: Advanced MapReduce & Spark Structured APIs

Môn học: Nhập môn dữ liệu lớn

Mã lớp: CQ2022/21

GVHD1: Nguyễn Ngọc Thảo

GVHD2: Lê Ngọc Thành

Trợ Giảng: Huỳnh Lâm Hải Đăng

Thành viên:	Phan Bá Đức	22120071
	Đặng Duy Lâm	22120182
	Nguyễn Quốc Thịnh	22120347
	Trần Đắc Thịnh	22120348

Thành phố Hồ Chí Minh, ngày 11 tháng 04 năm 2025

Mục lục

SECTION 1: Phân công thành viên nhóm	1
SECTION 2: Báo cáo chi tiết về bài làm	2
1. Tìm hiểu và hướng dẫn cài đặt Spark.....	2
2. Tìm hiểu và viết mã nguồn cho Task_2.1	5
3. Tìm hiểu và viết mã nguồn cho Task_2.2.....	8
4. Tìm hiểu và viết mã nguồn cho Task_2.3.....	9

SECTION 1: Phân công thành viên nhóm

MSSV	Họ và tên	Phân công công việc
22120071	Phan Bá Đức	Tổng hợp code, kiểm thử, viết báo cáo
22120182	Đặng Duy Lâm	Tìm hiểu và viết mã nguồn cho Task_2.1
22120347	Nguyễn Quốc Thịnh	Tìm hiểu và viết mã nguồn cho Task_2.3
22120348	Trần Đắc Thịnh	Tìm hiểu và viết mã nguồn cho Task_2.2

SECTION 2: Báo cáo chi tiết về bài làm

1. Tìm hiểu và hướng dẫn cài đặt Spark

1.1 Cài đặt python 3.10:

```
python3 --version # Kiểm tra phiên bản đang sử dụng

sudo apt update # Cài đặt các gói phụ thuộc
sudo apt install software-properties-common -y

#Thêm kho lưu trữ chứa các phiên bản Python cũ
sudo add-apt-repository ppa:deadsnakes/ppa -y
sudo apt update

# Cài đặt Python 3.10
sudo apt install python3.10 -y

# Kiểm tra cài đặt
python3.10 --version
ls /usr/bin/python3* # Kiểm tra các phiên bản Python hiện có
```

1.2 Đảm bảo có java 8/11/17:

```
java --version # kiểm tra phiên bản
```

- Vì các bạn đã tải Hadoop trước đó nên vì thế sẽ không cần kiểm tra cũng được.

1.3 Tải và cài Spark (chọn phiên bản từ 3.5 trở lên):

- Tải và giải nén spark: <https://spark.apache.org/downloads.html>
- Cách số 2 (sử dụng cmd / terminal):

```
wget <link tải spark>
tar -xvzf <file tải về>
```

Sau khi tải thì kiểm tra lại:

```
java -version
python3.10 --version
cd Downloads
ls
```

Di chuyển Spark vào thư mục /opt/

```
sudo mv spark-3.5.5-bin-hadoop3 /opt/spark
ls /opt/
```

Cấu hình biến môi trường:

```
nano ~/.bashrc
```

Thêm vào cuối file:

```
export SPARK_HOME=/opt/spark  
export PATH=$SPARK_HOME/bin:$PATH
```

Lưu file lại:

```
source ~/.bashrc
```

Chạy thử SPARK:

```
spark-shell
```

1.4 Cài PySpark:

Kiểm tra PySpark: pyspark

Cấu hình lại để PySpark dùng python 3.10: Trong file .bashrc ta tiến hành thêm dòng lệnh bên dưới rồi lưu lại.

```
export PYSARK_PYTHON=python3.10
```

Kiểm tra lại PySpark: pyspark

Gõ thử đoạn sau để kiểm tra:

```
rdd = sc.parallelize([1, 2, 3, 4])  
rdd.map(lambda x: x * 2).collect()
```

1.5 Tiếp tục với VSCODE:

- Cài Jupyter (*Vì code file .py debug khó, nên dùng Jupyter Notebook cho dễ.*)
- Chọn Python 3.10 trong VS Code

- Nhấn Ctrl + Shift + P
- Gõ Python: Select Interpreter
- Chọn Python 3.10.16 (/usr/bin/python3.10)

- Cài đặt PySpark:

```
sudo apt install python3.10-venv -y  
python3.10 -m ensurepip --upgrade  
python3.10 -m pip install --upgrade pip  
python3.10 -m pip install pyspark
```

- Test code với Spark:

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("MySimpleApp") \
    .getOrCreate()

sc = spark.sparkContext

rdd = sc.parallelize([1, 2, 3, 4])
print(rdd.map(lambda x: x * 2).collect())

spark.stop()
```

2. Tìm hiểu và viết mã nguồn cho Task_2.1

- Thành viên: Đặng Duy Lâm - 22120182
- Công cụ: Python + mrjob (MapReduce) + Hadoop
- Yêu cầu bài toán: Đề bài yêu cầu sử dụng thuật toán của sổ trượt 3 ngày để tính tổng số lượng bán ra cho mỗi loại hàng trong từng ngày.

a. Ý tưởng bài toán:

Tính tổng doanh thu của từng danh mục sản phẩm trong 3 ngày gần nhất (hôm nay và 2 ngày tiếp theo). Mỗi dòng dữ liệu là một đơn hàng, chỉ tính nếu đã giao thành công (status = shipped).

b. Lý do lựa chọn giải pháp:

MapReduce không giữ trạng thái toàn cục, do đó cần “đẩy” doanh thu của từng dòng vào trước 3 ngày liên tiếp, nhằm giả lập sliding window. Cách làm này đơn giản, phù hợp với nguyên lý của MapReduce: gom nhóm theo key và cộng tổng tại reducer.

c. Mã nguồn

```
from mrjob.job import MRJob
from datetime import datetime, timedelta
import csv
```

Nhập các thư viện cần dùng: mrjob để tạo job MapReduce, datetime để xử lý ngày, csv để tách dòng.

```
class Ex21(MRJob):
    def mapper(self, _, inp):
        global column_names

        # first line in csv file
        if inp.startswith("index"):
            column_names = inp.split(',')
            return
```

Nếu là dòng đầu (header), lưu lại tên cột để dùng sau, không xử lý.

```
row = list(csv.reader([inp]))[0]
date, category = "", ""
revenue = 0
is_shipped = False # only consider row with status = shipped
```

Đọc dòng dữ liệu thành danh sách, khởi tạo biến để xử lý.

```
for i in range(len(column_names)):
    col = column_names[i].strip().lower()
    row_val = row[i].strip()

    if col == 'date':
        date = row_val
        dt = datetime.strptime(date, '%m-%d-%y')
    elif col == 'category':
        category = row_val
    elif col == 'amount':
        if row[i].strip() != '':
            revenue = float(row_val)
    elif col == 'status':
        is_shipped = row_val.lower() == 'shipped'

if not is_shipped:
    return
```

Duyệt từng cột để lấy đúng thông tin cần dùng.

Nếu đơn hàng chưa được giao, bỏ qua.

```
tomorrow = dt + timedelta(days=1)
tomorrow_of_tomorrow = tomorrow + timedelta(days=1)

yield (dt.strftime('%d/%m/%Y'), category), revenue
yield (tomorrow.strftime('%d/%m/%Y'), category), revenue
yield (tomorrow_of_tomorrow.strftime('%d/%m/%Y'), category), revenue
```

Với mỗi đơn hàng, tạo 3 bản ghi tương ứng 3 ngày: hôm nay, mai và ngày kia.

```
def reducer(self, key, value):  
    report_date, category = key  
    yield (report_date, category, sum(value)), None
```

Cộng tất cả doanh thu theo cặp (ngày, danh mục) lại với nhau.

Kết quả: xuất ra file out.txt (đây là file raw)

```
["23/06/2022", "Top", 85808.0] null  
["23/06/2022", "Western Dress", 149805.0] null  
["23/06/2022", "kurta", 463975.0] null  
["24/04/2022", "Blouse", 14997.0] null  
["24/04/2022", "Bottom", 7850.0] null  
["24/04/2022", "Ethnic Dress", 10303.0] null  
["24/04/2022", "Saree", 5781.0] null  
["24/04/2022", "Set", 1025338.0] null  
["24/04/2022", "Top", 131976.0] null  
["24/04/2022", "Western Dress", 156369.0] null
```

Tiến hành viết thêm một file notebook (**format.ipynb**) để lọc và sắp xếp dữ liệu sau đó xuất ra output.csv theo yêu cầu đề bài

Chi tiết thông tin về file này đã được ghi đầy đủ trong format.ipynb nên không cần ghi trong báo cáo này

```
output.csv X  
F: > Code > Python > BigData_Lab2_Ex2.1 > output.csv > data  
1  report_date,category,revenue  
2  31/03/2022,Set,47217.0  
3  31/03/2022,Top,4065.0  
4  31/03/2022,Western Dress,2198.0  
5  31/03/2022,kurta,30533.0  
6  01/04/2022,Blouse,4516.0  
7  01/04/2022,Bottom,360.0  
8  01/04/2022,Ethnic Dress,11466.0  
9  01/04/2022,Set,394350.0  
10 01/04/2022,Top,52031.0  
11 01/04/2022,Western Dress,46438.0  
12 01/04/2022,kurta,195491.0  
13 02/04/2022,Blouse,7894.0  
14 02/04/2022,Bottom,1175.0  
15 02/04/2022,Ethnic Dress,14537.0
```

- **Chương trình thực thi thành công!**

3. Tìm hiểu và viết mã nguồn cho Task_2.2

- Thành viên: Trần Đức Thịnh - 22120348
- Công cụ: PySpark
- Bài toán yêu cầu đếm số lượng sản phẩm được bán đi trong mỗi tuần của mỗi loại sản phẩm bắt đầu tính từ ngày thứ 2. Ví dụ: Ta cần tính số lượng được bán đi của loại sản phẩm “Set” trong khoảng thời gian “25-04-2022” – “01-05-2022” để báo cáo cho ngày “02-05-2022”.
 - Ý tưởng bài toán**
 - Dữ liệu được lọc giữ lại các đơn có Status = Shipped và Qty > 0.
 - Với mỗi dòng, tính report_date là Thứ Hai gần nhất sau ngày đơn hàng (**nếu dữ liệu không đủ 7 ngày thì sẽ bị bỏ qua**)
 - Gom nhóm dữ liệu theo report_date và SKU, rồi tính tổng Qty.
 - Lý do cho giải pháp**
 - Do hạn chế về khối lượng dữ liệu và đơn giản hóa bài toán, nhóm chọn cách tiếp cận là gán đơn hàng về ngày Thứ Hai gần nhất sau nó, thay vì tính sliding window 7 ngày thật sự.
 - Cách làm này nhanh, dễ triển khai và vẫn phản ánh xu hướng tiêu thụ sản phẩm trên từng SKU theo tuần.
 - Mã nguồn** (chi tiết đã được trình bày trong file Lab02_Ex02.ipynb)

Kết quả

```
1 report_date,sku,total_quantity
2 11-04-2022,AN201-RED-M,1
3 11-04-2022,AN202-ORANGE-S,1
4 11-04-2022,AN205-YELLOW-S,2
5 11-04-2022,AN206-GREEN-M,1
6 11-04-2022,AN208-MUSTARD-M,1
7 11-04-2022,AN208-MUSTARD-XL,1
8 11-04-2022,AN209-BIEGE-XL,1
9 11-04-2022,AN213-BROWN-S,2
10 11-04-2022,BL008-61RED-B,1
11 11-04-2022,BL015-63PINK,1
12 11-04-2022,BL050-83RED-A,1
13 11-04-2022,BL079-87RED,1
14 11-04-2022,BL086-XXL,1
15 11-04-2022,BL096-S,1
16 11-04-2022,BL102-L,1
17 11-04-2022,BL102-M,1
18 11-04-2022,BL103-XS,1
19 11-04-2022,BL103-XXL,1
```

4. Tìm hiểu và viết mã nguồn cho Task_2.3

- Thành viên thực hiện: Nguyễn Quốc Thịnh – 22120347
- Công cụ: Pyspark
- Yêu cầu bài toán: Cho một tập các hình chữ nhật dưới dạng danh sách đỉnh (vertices), hãy tìm ra tất cả các **cặp hình có giao nhau** thực sự (tức là diện tích vùng giao > 0).
- Dữ liệu lưu trong file .parquet có dạng: <shape_id;vertices>. Trong đó:
 - **vertices** là danh sách 4 đỉnh biểu diễn một hình chữ nhật (theo thứ tự ngược chiều kim đồng hồ).
 - **shape_id**, nếu bị thiếu (NULL), cần được tạo lại để định danh các hình.
- a. Ý tưởng bài toán
 - Trước tiên, kiểm tra tất cả các hình xem cạnh của chúng có song song với trục tọa độ không (kiểm tra song song với trục X, Y).
 - Sau đó sinh các cặp hình khác nhau bằng **crossJoin**, bỏ các cặp trùng.
 - Với mỗi cặp hình, tính:
 - Chiều dài, chiều rộng của từng hình.
 - Chiều dài, chiều rộng của khung bao ngoài chứa được cả 2 hình.

- Nếu chiều dài hoặc chiều rộng của khung bao nhỏ hơn tổng chiều dài hoặc rộng của 2 hình → 2 hình có giao nhau.
- b. Lý do lựa chọn giải pháp
 - Kiểm tra song song với trục giúp đảm bảo hình chữ nhật dễ xử lý hơn (không cần dùng hình học vector).
 - Dùng toán học đơn giản (so sánh tổng chiều dài với kích thước khung bao) để xác định giao nhau.
 - Cách làm này hiệu quả, tận dụng tối đa khả năng xử lý song song của Spark

Kết quả

```
+-----+-----+
|shape_1|shape_2|
+-----+-----+
|      0|      41|
|      0|     143|
|      0|     198|
|      0|     430|
|      0|     473|
|      0|     490|
|      0|     496|
|      0|     605|
|      0|     637|
|      0|     729|
|      0|     818|
|      1|     118|
|      1|     160|
|      1|     186|
|      1|     220|
|      1|     247|
|      1|     256|
|      1|     377|
|      1|     444|
|      1|     809|
+-----+-----+
only showing top 20 rows
```